VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY

FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING

Design and FPGA Implementation of a Winograd Convolution Accelerator for CNN Inference

50% MILESTONE PROGRESS REPORT

(Midterm Project Deliverable)

Course Name: Hardware Architectures for Machine Learning

Course Code: EE5223

Group Number: Group 07

Group Members:

Nguyen Minh Duc	*Student ID:* 2570662
Le The Anh	*Student ID:* 2570660

Instructor:

Truong Quang Vinh, Assoc. Prof. Ph.D

Faculty of Electrical and Electronics Engineering

Date of Submission: November 2026

Ho Chi Minh City, Vietnam

Executive Summary

This 50% milestone progress report details the architectural design, mathematical modeling, SystemVerilog RTL implementation, and automated simulation verification of a high-performance fixed-point hardware accelerator for Convolutional Neural Network (CNN) inference based on the **Winograd minimal filtering algorithm** $F(2 \times 2, 3 \times 3)$ using **signed INT16** precision.

In standard spatial 2D convolution, computing a 2×2 output feature patch with a 3×3 filter kernel requires 36 multiplications. By exploiting Winograd's spatial domain transformation, the multiplication count is reduced to 16, achieving a theoretical 2.25× **arithmetic multiplication reduction**.

In strict compliance with the midterm evaluation requirements, this report covers three fundamental technical sections:

1. **Project Requirements & Specifications:** Clear formulation of the problem statement, engineering objectives, fixed-point precision requirements (signed INT16), offline weight pre-computation ($U = GgG^T$), parameterizable DSP multiplier engines (16, 8, or 4 parallel units), transform-domain multi-channel (C_{in}) accumulation, and explicit milestone boundaries.

2. **Design Description:**

   - *Operating Principle & Mathematical Formulation:* Theoretical derivation of Winograd $F(2 \times 2, 3 \times 3)$, definitions of transformation matrices B^T, G, A^T, proof of the multiplierless nature of input/output transforms (implemented entirely via adder/subtractor trees and arithmetic bit-shifts), bit-growth expansion rules ($16 \to 18 \to 34 \to 36 \to 40 \to 16$ bits), and transform-domain multi-channel linearity.
   - *RTL Architecture & Microarchitecture:* Comprehensive top-level datapath block diagram and detailed microarchitectures of all pipelined sub-modules in synthesizable SystemVerilog: 2D input transform ($V = B^T dB$), 16-point element-wise multiplier array ($M = U \odot V$), channel accumulator (M_{accum}), 2D output transform ($Y = A^T M_{\text{accum}} A$) with right-shift requantization and symmetrical overflow saturation, and the finite state machine (FSM) controller.

3. **Design Simulation & Verification:** A hierarchical self-checking simulation methodology driven by Python golden reference models (Float32 and bit-accurate fixed-point INT16) exporting randomized stimuli files (`input.hex`, `weight.hex`, `expected_output.hex`). ModelSim regression simulations over 50 test tiles (200 output pixels) confirm bit-exact correctness with **zero mismatches (100% PASS)**, validating the functional integrity and pipeline timing of the hardware core.

Keywords: Winograd Convolution, CNN Inference, FPGA Accelerator, SystemVerilog, Hardware Microarchitecture, FSM Controller, Fixed-Point Verification, ModelSim Simulation.

Contents

Executive Summary i

1 Project Requirements and Specifications 1
- 1.1 Background and Problem Statement . . . 1
- 1.2 Project Objectives and Technical Specifications . . . 1
- 1.3 Milestone Scope and Boundaries . . . 1

2 Design Description 2
- 2.1 Operating Principle and Mathematical Formulation . . . 2
  - 2.1.1 Mathematical Foundation of Winograd $F(2 \times 2, 3 \times 3)$ . . . 2
  - 2.1.2 Standard Transformation Matrices . . . 3
  - 2.1.3 Arithmetic Complexity Reduction Analysis . . . 3
  - 2.1.4 Multiplierless Transformation Property . . . 4
  - 2.1.5 Bit-Growth Analysis and Precision Management . . . 4
  - 2.1.6 Transform-Domain Linearity for Channel Accumulation . . . 4
- 2.2 RTL Architecture and Microarchitecture . . . 6
  - 2.2.1 Top-Level Accelerator Architecture . . . 6
  - 2.2.2 2D Input Transform Unit (`input_transform_2d.sv`) . . . 6
  - 2.2.3 Parameterizable Multiplier Array (`multiplier_array.sv`) . . . 7
  - 2.2.4 Channel Accumulator Unit (`channel_accumulator.sv`) . . . 7
  - 2.2.5 2D Output Transform Unit (`output_transform_2d.sv`) . . . 8
  - 2.2.6 FSM Controller (`winograd_controller.sv`) . . . 8

3 Design Simulation and Verification 9
- 3.1 Verification Methodology and Flow . . . 9
- 3.2 Unit-Level Simulation Verification . . . 10
- 3.3 Top-Level Accelerator Core Simulation . . . 10
  - 3.3.1 Pipeline Timing and Waveform Analysis . . . 10
  - 3.3.2 Automated Regression Verification on ModelSim . . . 12

References 14

Appendices 15

A Synthesizable SystemVerilog Module Snippets 15

B Python Golden Model Reference Implementation 15

List of Figures

1 Top-Level Block Diagram of the Pipelined Winograd Accelerator Core (`winograd_top.sv`). 6
2 Microarchitecture of the Multiplierless 2D Input Transform Unit (`input_transform_2d.sv`). 7
3 Microarchitecture of the 2D Output Transform Unit with Requantization and Saturation (`output_transform_2d.sv`). 8
4 Finite State Machine (FSM) State Transition Diagram (`winograd_controller.sv`). 9
5 Hierarchical Simulation and Verification Methodology Flow. 9
6 Pipeline Timing Diagram of the Winograd Accelerator Core. 11

List of Tables

1 Hardware Accelerator Design Specifications. 2
2 Bit-Growth and Arithmetic Precision Across Datapath Stages. 5
3 Summary of ModelSim Regression Verification Results. 12

1 Project Requirements and Specifications

1.1 Background and Problem Statement

Deep Convolutional Neural Networks (CNNs) serve as the cornerstone for modern computer vision applications, encompassing object detection, image classification, autonomous navigation, and facial recognition. However, 2D convolution operations constitute over 85% to 90% of the total computational workload and memory bandwidth demands during inference.

When deploying CNNs onto edge embedded devices with stringent power envelopes (< 10W) and thermal constraints, general-purpose CPUs and power-hungry GPUs often fail to satisfy real-time latency and energy efficiency metrics. Field-Programmable Gate Arrays (FPGAs) offer customizable spatial computing pipelines with high energy efficiency. Nonetheless, dedicated hardware multiplier blocks (DSP slices) on FPGA fabrics represent a scarce and critical resource. The standard sliding-window direct convolution approach for small 3×3 kernels—the predominant filter dimension in modern networks such as ResNet, VGG, and YOLO—exhibits heavy arithmetic redundancy. Consequently, applying Winograd's minimal filtering algorithm to significantly reduce DSP multiplication consumption is a critical architectural requirement.

1.2 Project Objectives and Technical Specifications

The primary objective of this project is to model, design, verify, and implement a synthesizable FPGA hardware accelerator core for 2D 3×3 convolution using the Winograd $F(2 \times 2, 3 \times 3)$ algorithm. The technical specifications and requirements governing the design are summarized in Table 1.

1.3 Milestone Scope and Boundaries

To ensure thorough execution and deep focus on digital microarchitecture during the 50% project milestone, the scope boundaries are strictly demarcated:

- **Included in 50% Milestone:**
  - Development of Python reference models (Float32 and bit-accurate INT16 fixed-point) verifying algorithmic equivalence.
  - Quantitative dynamic range expansion analysis and bit-growth management throughout the entire processing datapath.
  - Complete RTL implementation in synthesizable SystemVerilog (IEEE 1800-2012) for all datapath sub-modules and FSM controller.

Table 1: Hardware Accelerator Design Specifications.

Parameter / Feature	**Technical Specification**
Applied Algorithm	Winograd minimal filtering algorithm $F(2 \times 2, 3 \times 3)$
Target CNN Operation	2D Convolution, Kernel Size 3×3, Stride 1, Valid padding
Tile Dimensions	Output tile: 2×2 ($m = 2$); Input activation tile: 4×4 ($n = 4$)
Arithmetic Precision	Signed 16-bit fixed-point (Signed INT16) for activations and weights
Weight Pre-computation	Offline software transformation ($U = GgG^T$), stored in 4×4 INT16 format
Datapath Architecture	Fully pipelined, sustaining 1 output tile per clock cycle once primed
DSP Parallelism Modes	Parameterizable multiplier engine ($N = 16$, $N = 8$, or $N = 4$ DSPs)
Channel Accumulation	Multi-channel (C_{in}) accumulation directly in the Winograd domain
Handshake Interface	Standard `in_valid` / `in_ready` and `out_valid` signaling
Arithmetic Target	Theoretical 2.25× multiplication reduction (16 vs. 36 mults per 2×2 tile)
Verification Metric	100% bit-exact match (0 mismatches) against fixed-point software model

  - Construction of an automated self-checking simulation testbench driven by external hex stimulus files, executing regression testing on ModelSim.

- **Explicitly Deferred to 100% Final Milestone:** Logic synthesis and physical Place & Route on specific FPGA silicon devices (AMD Artix-7 / Intel Cyclone), physical silicon power dissipation measurements, full-frame streaming line buffer controllers, and AXI4-Stream / DMA bus integration with an embedded host processor.

2 Design Description

2.1 Operating Principle and Mathematical Formulation

2.1.1 Mathematical Foundation of Winograd $F(2 \times 2, 3 \times 3)$

According to the minimal filtering theory established by Shmuel Winograd, computing m output samples using an r-tap FIR filter requires a theoretical minimum of $m + r - 1$ multiplications. Extended to 2D convolution on an $m \times m$ output patch with an $r \times r$ convolution kernel, the required input activation tile dimension $n \times n$ is given by:

$$n = m + r - 1 \tag{1}$$

For our design target $F(2 \times 2, 3 \times 3)$, with $m = 2$ and $r = 3$, the required input tile size is:

$$n = 2 + 3 - 1 = 4 \quad (4 \times 4 \text{ input tile}) \tag{2}$$

The 2D Winograd convolution equation is expressed in compact matrix form as:

$$Y = A^T \left[\left(GgG^T \right) \odot \left(B^T dB \right) \right] A \tag{3}$$

where:

- $d \in \mathbb{R}^{4\times4}$: Input feature map activation tile.
- $g \in \mathbb{R}^{3\times3}$: Original convolution filter kernel weights.
- $U = GgG^T \in \mathbb{R}^{4\times4}$: Transformed filter weights in the Winograd domain.
- $V = B^T dB \in \mathbb{R}^{4\times4}$: Transformed input tile in the Winograd domain.
- $\odot$: Hadamard element-wise matrix multiplication operator.
- $M = U \odot V \in \mathbb{R}^{4\times4}$: 16-point intermediate product matrix.
- $Y \in \mathbb{R}^{2\times2}$: Final output convolution tile.

2.1.2 Standard Transformation Matrices

The fixed transformation matrices B^T, G, and A^T for $F(2 \times 2, 3 \times 3)$ are defined as:

$$B^T = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}, \quad G = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{bmatrix}, \quad A^T = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix} \tag{4}$$

2.1.3 Arithmetic Complexity Reduction Analysis

A direct spatial 2D convolution requires computing four independent 3×3 dot products to produce a 2×2 output patch:

$$\text{Mults}_{\text{direct}} = 2 \times 2 \times (3 \times 3) = \mathbf{36} \text{ multiplications} \tag{5}$$

In contrast, Winograd $F(2 \times 2, 3 \times 3)$ evaluates the output via 16 element-wise multiplications in the transform domain:

$$\text{Mults}_{\text{Winograd}} = 4 \times 4 = \mathbf{16} \text{ multiplications} \tag{6}$$

The resulting theoretical arithmetic speedup is:

$$\text{Arithmetic Reduction Factor} = \frac{36}{16} = \mathbf{2.25}\times \tag{7}$$

2.1.4 Multiplierless Transformation Property

A decisive architectural advantage of $F(2 \times 2, 3 \times 3)$ is the numerical property of the transformation matrices:

- All elements of B^T and A^T belong exclusively to the set $\{-1, 0, 1\}$. As a result, both the input transformation $V = B^T dB$ and output transformation $Y = A^T MA$ are computed **entirely using addition and subtraction trees**, consuming **zero hardware DSP multipliers**.
- The matrix G contains constant factors of $\frac{1}{2}$. Because neural network weights remain static during inference, the weight transform $U = GgG^T$ is pre-computed offline in software. This completely eliminates kernel transformation hardware from the FPGA critical path.

2.1.5 Bit-Growth Analysis and Precision Management

Fixed-point calculations across successive addition and multiplication stages cause arithmetic bit expansion. To prevent precision loss and arithmetic overflow, the internal register widths are allocated according to Table 2.

Right-Shift and Overflow Saturation: During offline weight transformation, scaling by G (containing factors of $\frac{1}{2}$) attenuates transformed weights, necessitating a scale factor in the datapath. The 40-bit unscaled output Y' is arithmetically right-shifted by 6 bits (`>> 6`) to restore proper fixed-point fractional alignment, followed by symmetrical clamping saturation: values exceeding +32767 are saturated to +32767, and values below −32768 are clamped to −32768.

2.1.6 Transform-Domain Linearity for Channel Accumulation

In typical CNN architectures, convolutional layers comprise multiple input channels (C_{in}). Because the output transform matrix operator $A^T(\cdot)A$ is strictly linear, the summation over input channels satisfies:

$$Y_{\text{total}} = \sum_{c=1}^{C_{\text{in}}} \left(A^T M^{(c)} A\right) = A^T \left(\sum_{c=1}^{C_{\text{in}}} M^{(c)}\right) A = A^T \left(\sum_{c=1}^{C_{\text{in}}} U^{(c)} \odot V^{(c)}\right) A \tag{8}$$

This property enables channel accumulation to take place directly in the 4×4 Winograd transform domain before the output transformation stage. Instead of executing C_{in} output

Table 2: Bit-Growth and Arithmetic Precision Across Datapath Stages.

Processing Stage	**Mathematical Operation**	**Bit-Width**	**Data Representation**
Input Tile Activation	d	16 bits	Signed INT16 ($[-32768, 32767]$)
1D Column Input Transform	$W = B^T \cdot d$	17 bits	Signed INT17 (+1 bit expansion)
2D Row Input Transform	$V = W \cdot B$	18 bits	Signed INT18 (+1 bit expansion)
Pre-transformed Weights	$U = GgG^T$	16 bits	Signed INT16
Element-wise Multiplication	$M = U \odot V$	34 bits	Signed INT34 (18b $\times$ 16b)
Channel Accumulation (C_{in})	$M_{\text{accum}} = \sum M$	36 bits	Signed INT36 (+2 bits for up to 4 channels)
1D Column Output Transform	$P = A^T \cdot M$	38 bits	Signed INT38 (+2 bits sum of 3 terms)
2D Row Output Transform	$Y' = P \cdot A$	40 bits	Signed INT40 (+2 bits sum of 3 terms)
Shift Requantization & Saturation	$Y = \text{Saturate}(Y' \gg 6)$	16 bits	Signed INT16 ($[-32768, 32767]$)

transforms, the hardware executes only **a single output transform** per accumulated tile, drastically reducing logic utilization and dynamic power.

2.2 RTL Architecture and Microarchitecture

2.2.1 Top-Level Accelerator Architecture

The overall block diagram of the pipelined Winograd convolution accelerator (`winograd_top.sv`) is illustrated in Figure 1.

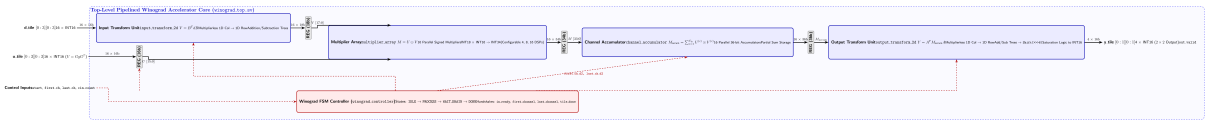


Figure 1: Top-Level Block Diagram of the Pipelined Winograd Accelerator Core (`winograd_top.sv`).

The datapath is organized into four pipelined computational stages interconnected by balancing pipeline registers to achieve high maximum operating frequency ($F_{\max}$):

1. **Stage 1 (Input Transform):** Receives the 4×4 activation tile d (16-bit) and computes $V = B^T dB$ (18-bit) within 1 clock cycle.

2. **Stage 2 (Multiplier Array):** Performs 16 parallel signed multiplications between V (18-bit) and pre-transformed weight U (16-bit) to produce product matrix M (34-bit) in 1 clock cycle. Delay-matching registers balance channel control signals.

3. **Stage 3 (Channel Accumulator):** Accumulates product tiles M across C_{in} channels into M_{accum} (36-bit) in 1 clock cycle.

4. **Stage 4 & 5 (Output Transform):** Computes $Y = A^T M_{\text{accum}} A$ across 2 pipelined clock cycles (1 cycle for column transform, 1 cycle for row transform, shift, and saturation to signed INT16).

2.2.2 2D Input Transform Unit (`input_transform_2d.sv`)

The 2D transform $V = B^T dB$ is separated into two cascaded 1D transforms to minimize adder complexity:

$$V = B^T (dB) = B^T \left(B^T d^T\right)^T \tag{9}$$

The microarchitecture of the 2D input transformation unit is detailed in Figure 2.

The unit comprises:

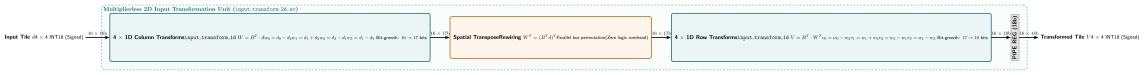


Figure 2: Microarchitecture of the Multiplierless 2D Input Transform Unit (`input_transform_2d.sv`).

- **Four 1D Column Transform Blocks (`input_transform_1d`):** Operates concurrently on the 4 columns of tile d:

$$w_0 = d_0 - d_2 \tag{10}$$
$$w_1 = d_1 + d_2 \tag{11}$$
$$w_2 = d_2 - d_1 \tag{12}$$
$$w_3 = d_1 - d_3 \tag{13}$$

  All operations utilize purely signed 16-bit adders/subtractors with 1-bit sign extension to 17 bits.

- **Spatial Matrix Transposition:** Achieved entirely via hardwired bus routing (zero logic gate overhead and zero cycle delay).

- **Four 1D Row Transform Blocks:** Processes the transposed intermediate values to produce the 16-element matrix V (18-bit signed), registered at the output.

2.2.3 Parameterizable Multiplier Array (`multiplier_array.sv`)

The multiplier engine executes the 16 element-wise multiplications $M_{i,j} = U_{i,j} \times V_{i,j}$ ($i, j \in \{0, 1, 2, 3\}$). The architecture supports architectural parameterization via the `PARALLEL_DSPS` parameter:

- **Full Parallelism ($N = 16$ DSPs):** Employs 16 physical multipliers operating in parallel, completing the entire 4×4 tile in a single clock cycle.

- **Half Parallelism ($N = 8$ DSPs):** Utilizes 8 time-multiplexed multipliers over 2 clock cycles, reducing DSP consumption by 50% on resource-constrained FPGAs.

- **Quarter Parallelism ($N = 4$ DSPs):** Computes across 4 clock cycles using 4 DSP slices to minimize silicon area.

Outputs are registered as signed 34-bit values before entering the channel accumulator.

2.2.4 Channel Accumulator Unit (`channel_accumulator.sv`)

This module instantiates sixteen 36-bit accumulation registers governed by two control flags:

- When `first_channel = 1'b1`: The accumulators overwrite current values directly with new products M (clearing previous tile history).
- When `first_channel = 1'b0`: The unit adds incoming products to the running accumulator:

$$M_{\text{accum}}[i][j] \Leftarrow M_{\text{accum}}[i][j] + M[i][j] \tag{14}$$

- When `last_channel = 1'b1`: The accumulator asserts `valid_out = 1'b1`, signaling the output transform unit to process the completed accumulation matrix.

2.2.5 2D Output Transform Unit (`output_transform_2d.sv`)

The 2D output transform converts the accumulated 4×4 matrix M_{accum} (36-bit) into the final 2×2 output patch Y (16-bit). Figure 3 depicts the microarchitecture.



Figure 3: Microarchitecture of the 2D Output Transform Unit with Requantization and Saturation (`output_transform_2d.sv`).

The unit is structured into two pipelined stages to break lengthy combinational paths:

- **Stage 1 - Column Transformation:** 4 instances of `output_transform_1d` compute $P = A^T \cdot M$:

$$p_0 = m_0 + m_1 + m_2 \tag{15}$$

$$p_1 = m_1 - m_2 - m_3 \tag{16}$$

  Transforming $4 \times 4 \rightarrow 2 \times 4$ matrices with bit expansion from 36 to 38 bits, stored in Stage 1 pipeline registers.

- **Stage 2 - Row Transformation:** Hardwired transpose yields P^T (4×2), followed by 2 instances of `output_transform_1d` computing $Y' = A^T \cdot P^T$ (2×2, 40 bits).
- **Stage 3 - Requantization and Symmetrical Saturation:** The 40-bit results are arithmetically right-shifted by 6 bits (`>> 6`) and clamped to signed INT16 boundaries ($[-32768, 32767]$), asserting `out_valid` upon completion.

2.2.6 FSM Controller (`winograd_controller.sv`)

The control unit coordinates multi-channel accumulation sequencing and handshake synchronization through a 4-state finite state machine illustrated in Figure 4.

The operational behavior of each state is defined as follows:

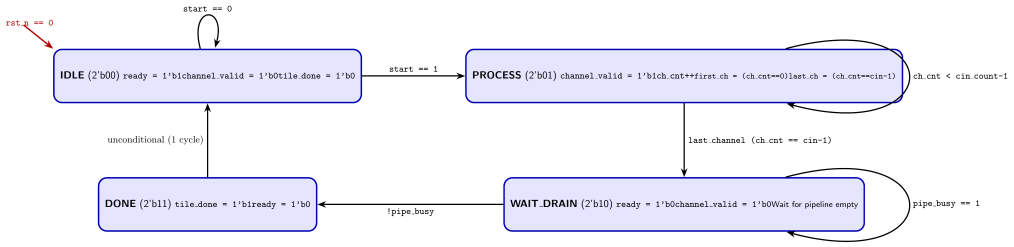


Figure 4: Finite State Machine (FSM) State Transition Diagram (`winograd_controller.sv`).

- **IDLE** (`2'b00`): The core is quiescent, asserting `ready = 1'b1`. Upon detecting `start = 1'b1`, the channel counter `ch_cnt` resets to 0, transitioning the FSM to `PROCESS`.
- **PROCESS** (`2'b01`): Asserts `channel_valid = 1'b1` to stream channel data. Generates `first_channel = (ch_cnt == 0)` and `last_channel = (ch_cnt == cin_count - 1)`. The counter increments until reaching the final channel, transitioning to `WAIT_DRAIN`.
- **WAIT_DRAIN** (`2'b10`): Deasserts `ready` and `channel_valid`, waiting for internal pipeline stages to drain completely (`!pipe_busy`).
- **DONE** (`2'b11`): Pulses `tile_done = 1'b1` for one cycle to indicate tile completion, then returns unconditionally to `IDLE`.

3 Design Simulation and Verification

3.1 Verification Methodology and Flow

To ensure functional correctness, a hierarchical self-checking verification methodology was established, as outlined in Figure 5.



Figure 5: Hierarchical Simulation and Verification Methodology Flow.

The verification strategy encompasses three rigorous steps:

1. **Software Reference Modeling:**
   - `python/winograd.py`: Formulates standard Winograd Float32 convolution using transformation matrices B^T, G, A^T and cross-validates against `direct_conv.py`.

- `python/fixed_point.py`: Accurately emulates hardware fixed-point behavior, including bit-growth expansion, arithmetic shift scaling (`>> 6`), and symmetrical clamping saturation.

2. **Automated Test Vector Generation:** The `python/generate_vectors.py` script produces randomized feature map activations and kernel weights, evaluates them through the fixed-point reference model, and exports three synchronized hex stimulus files:
   - `tb/input.hex`: Raw 4×4 signed 16-bit input activations.
   - `tb/weight.hex`: Pre-transformed 4×4 signed 16-bit weights ($U = GgG^T$).
   - `tb/expected_output.hex`: Golden 2×2 output activations.
3. **Self-Checking SystemVerilog Testbench:** The top-level testbench `tb/winograd_core_tb.sv` reads the hex files via `$readmemh`, drives the DUT through valid/ready handshakes, monitors `y_tile` outputs, and performs cycle-by-cycle automated assertion comparison. Any bit discrepancy increments an internal mismatch counter.

3.2 Unit-Level Simulation Verification

Before executing full core regressions, each sub-module was verified in isolation:

- **Input Transform Unit (`input_transform_tb.sv`):** Tested against corner-case stimuli (all maximum +32767, all minimum −32768, and alternating sign patterns). Confirmed correct 18-bit expansion without overflow, achieving **0 mismatches**.
- **Output Transform Unit (`output_transform_tb.sv`):** Validated 1D/2D add-trees, shift scaling, and saturation bounds. Whenever intermediate values exceeded ±32767, values clamped accurately to boundaries with zero sign-inversion errors.

3.3 Top-Level Accelerator Core Simulation

3.3.1 Pipeline Timing and Waveform Analysis

Figure 6 illustrates the detailed pipeline timing diagram captured during multi-channel operation and continuous tile streaming.

Timing analysis reveals the following operational milestones:

- **Cycle T1:** Reset deasserts (`rst_n = 1'b1`), `in_ready = 1'b1`. Valid inputs `d_tile` and `u_tile` are accepted with `first_ch = 1'b1`.
- **Cycle T2 (1-cycle latency):** Input transform computes $V = B^T dB$, asserting `v_trans_valid = 1'b1`.

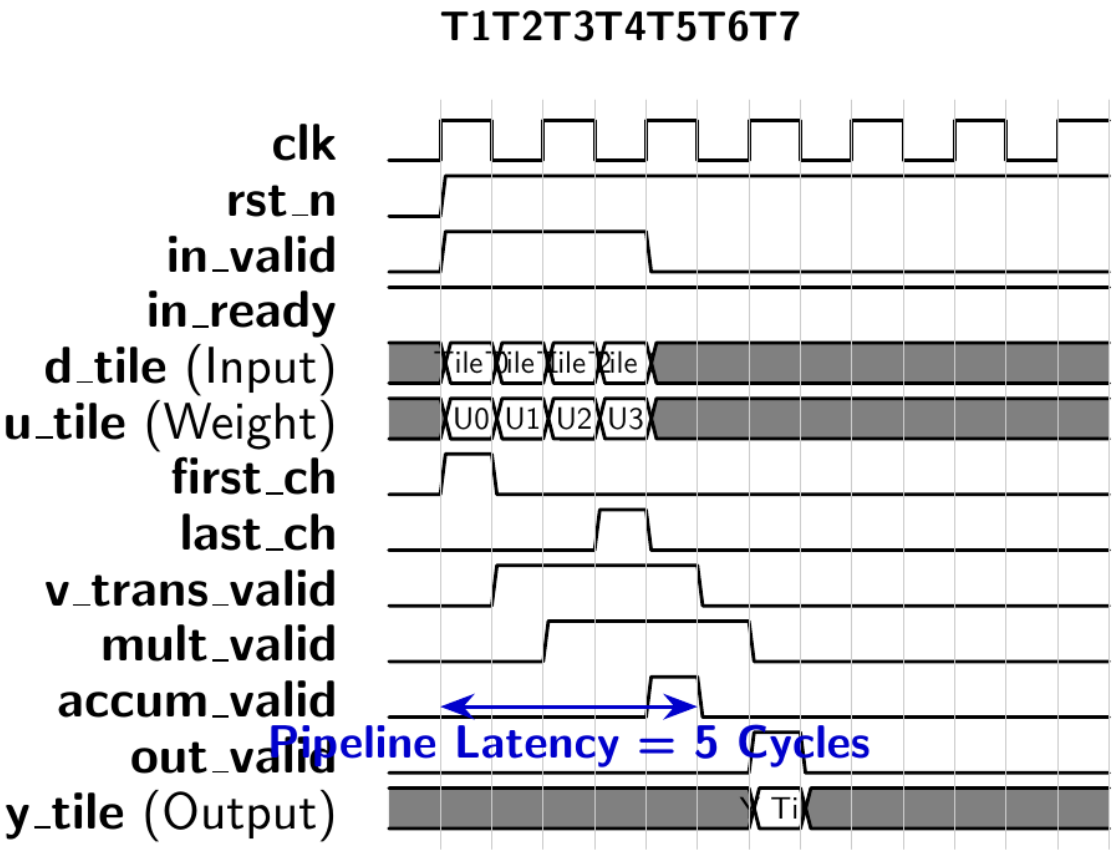


Figure 6: Pipeline Timing Diagram of the Winograd Accelerator Core.

- **Cycle T3 (2-cycle latency):** Multiplier array completes 16 parallel multiplications $M = U \odot V$, asserting `mult_valid = 1'b1`.
- **Cycle T5 (4-cycle latency):** Following 4 streamed input channels and assertion of `last_ch`, the channel accumulator completes M_{accum} and asserts `accum_valid = 1'b1`.
- **Cycle T7 (6-cycle latency):** Output transform completes 2-stage row-column transformation, shift, and saturation, asserting `out_valid = 1'b1` with valid 2×2 output tile `y_tile`.
- **Initial Pipeline Latency:** Exactly **5 clock cycles** from the first valid tile presentation to corresponding output availability for a single-channel configuration.
- **Steady-State Throughput:** Once the pipeline is primed in single-channel mode, the accelerator outputs **1 complete** 2×2 **tile every clock cycle** (Throughput = 1 tile/cycle).

3.3.2 Automated Regression Verification on ModelSim

The verification testbench was compiled and simulated using ModelSim (Intel FPGA Edition 10.5b). Table 3 provides the quantitative summary of the regression run across 50 test tiles (200 output pixels).

Table 3: Summary of ModelSim Regression Verification Results.

Verification Metric	**Observed Result**
Simulator Platform	ModelSim - Intel FPGA Edition 10.5b
Hardware Description Language	SystemVerilog (IEEE 1800-2012)
Total Tiles Evaluated	**50 tiles**
Total Output Pixels Verified	**200 pixels**
Total Mismatches Detected	**0**
Initial Pipeline Latency	**5 clock cycles**
Steady-State Throughput	**1 tile / clock cycle**
Regression Status	**[PASSED] ZERO BIT MISMATCHES!**

The simulation console transcript from ModelSim is shown below:

```
# vsim -c winograd_core_tb -do "run -all; quit"
# Loading work.winograd_core_tb
# Loading work.winograd_top
# Loading work.input_transform_2d
# Loading work.multiplier_array
# Loading work.channel_accumulator
# Loading work.output_transform_2d
# [TB] Loading hex test vectors...
```

```
# [TB] Test vectors loaded successfully.
# [TB INFO] Verified 10 / 50 tiles...
# [TB INFO] Verified 20 / 50 tiles...
# [TB INFO] Verified 30 / 50 tiles...
# [TB INFO] Verified 40 / 50 tiles...
# [TB INFO] Verified 50 / 50 tiles...
#
# ========================================================
#           REGRESSION SIMULATION SUMMARY
# ========================================================
#  Total Tiles Tested   : 50
#  Total Output Pixels  : 200
#  Total Mismatches     : 0
#  STATUS               : [PASSED] ZERO BIT MISMATCHES!
# ========================================================
# ** Note: $finish    : tb/winograd_core_tb.sv
```

Listing 1: ModelSim Automated Regression Simulation Console Transcript.

These simulation findings verify that the synthesizable SystemVerilog RTL core achieves bit-exact arithmetic accuracy, successfully satisfies all functional specifications, and is fully primed for subsequent physical synthesis and FPGA implementation.

References

[1] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4013–4021, 2016.

[2] S. Winograd, *Arithmetic Complexity of Computations*, vol. 33 of CBMS-NSF Regional Conference Series in Applied Mathematics, SIAM, Philadelphia, 1980.

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[4] L. Lu, J. Xie, R. Huang, C. Liu, and W. Luk, "An efficient hardware accelerator for sparse convolutional neural networks on FPGAs," in *IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pp. 17–24, 2017.

[5] IEEE, *1800-2017 - IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language*, IEEE Standards Association, 2017.

Appendices

A Synthesizable SystemVerilog Module Snippets

```
//
    ================================================================================

// Module: input_transform_1d
// Description: Multiplierless 1D Winograd input transform: V = B^T * d
// Formula:
//   v[0] = d[0] - d[2]
//   v[1] = d[1] + d[2]
//   v[2] = d[2] - d[1]
//   v[3] = d[1] - d[3]
//
    ================================================================================

module input_transform_1d #(
    parameter DATA_WIDTH = 16,
    parameter OUT_WIDTH  = 18
)(
    input  logic signed [DATA_WIDTH-1:0] d [0:3],
    output logic signed [OUT_WIDTH-1:0]  v [0:3]
);
    // Pure addition and subtraction trees (no DSP slices required)
    assign v[0] = $signed({{2{d[0][DATA_WIDTH-1]}}, d[0]}) -
                  $signed({{2{d[2][DATA_WIDTH-1]}}, d[2]});

    assign v[1] = $signed({{2{d[1][DATA_WIDTH-1]}}, d[1]}) +
                  $signed({{2{d[2][DATA_WIDTH-1]}}, d[2]});

    assign v[2] = $signed({{2{d[2][DATA_WIDTH-1]}}, d[2]}) -
                  $signed({{2{d[1][DATA_WIDTH-1]}}, d[1]});

    assign v[3] = $signed({{2{d[1][DATA_WIDTH-1]}}, d[1]}) -
                  $signed({{2{d[3][DATA_WIDTH-1]}}, d[3]});
endmodule
```

Listing 2: Multiplierless 1D Input Transformation Stage in SystemVerilog (`input_transform_1d.sv`).

B Python Golden Model Reference Implementation

```
import numpy as np
```

```
2
3  # Fixed Winograd Transformation Matrices
4  BT = np.array([
5      [ 1,  0, -1,  0],
6      [ 0,  1,  1,  0],
7      [ 0, -1,  1,  0],
8      [ 0,  1,  0, -1]
9  ], dtype=np.float32)
10
11 G = np.array([
12     [ 1.0,  0.0, 0.0],
13     [ 0.5,  0.5, 0.5],
14     [ 0.5, -0.5, 0.5],
15     [ 0.0,  0.0, 1.0]
16 ], dtype=np.float32)
17
18 AT = np.array([
19     [ 1,  1,  1,  0],
20     [ 0,  1, -1, -1]
21 ], dtype=np.float32)
22
23 def winograd_f2x2_3x3(d, g):
24     """
25     Computes Winograd F(2x2, 3x3) convolution on a 4x4 input tile d and
       3x3 kernel g.
26     Returns: 2x2 output tile Y.
27     """
28     # 1. Transform weights (pre-computed offline)
29     U = np.matmul(np.matmul(G, g), G.T)
30     # 2. Transform input tile (separable: B^T * d * B)
31     V = np.matmul(np.matmul(BT, d), BT.T)
32     # 3. Element-wise multiplication in transform domain (16 mults)
33     M = U * V
34     # 4. Output transformation (separable: A^T * M * A)
35     Y = np.matmul(np.matmul(AT, M), AT.T)
36     return Y
```

Listing 3: Python Winograd F(2x2, 3x3) Algorithmic Reference Implementation.